BURRA SEC

Centrifuge Create3 Gate Audit Report

Prepared by Burra Security

Version 1.0

Researchers

Alex Filippov

fuzious

Taridoku

September 1, 2026

Contents

1 About Burra Security

Burra Security offers security auditing and advisory services with a special focus on cross-chain and interoperability protocols and their integrations. Learn about us at burrasec.com.

2 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any vulnerabilities. Subsequent security reviews, bug bounty programs, and on-chain monitoring are recommended.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

The in-scope code is the Create3 Gate: `DeployGate`, a single ownerless contract that splits a CREATE3 deployment into two phases. A validator commits what may be deployed, at which addresses, in what order and by whom; any executor it named then deploys exactly that, and nothing else.

The `Create3` library performs CREATE3 in the gate itself, CREATE2-ing a minimal proxy whose only job is to `CREATE` the payload, so an address covers the deployer and the salt but never the init code. `DeployGate` derives that salt as `namespaceSalt(validator, salt) = keccak256(validator, salt)`, giving every account a namespace no other account can collide with or block. Commitments bind each salt to `keccak256(initCodeHash, index)` under a per-id generation, and a per-commitment cursor enforces the committed order.

One `validate` call commits an entire deployment regardless of contract count, so a cold validator or Safe signs once per chain while a warmer executor key sends the rest — an executor gains no privilege beyond reproducing the committed init code at the committed address, or reverting. `setDelegate` lets a warm key commit on a cold validator's behalf, one level deep and revocable in one call, and re-committing under an id replaces it whole, so whatever the new commitment omits becomes undeployable.

The gate takes no constructor arguments, holds no privileged state, and holds no authority over what it deploys, and it is deployed through CREATE2 rather than CREATE3 so that its address covers its own code. Nothing chain-specific enters the address derivation, so a namespace is the same set of addresses on every network, and `addressOf(validator, salt)` precomputes them before anything exists anywhere — which is what lets a deployment of 56 contracts across eleven mainnets be reviewed as one commitment rather than eleven.

5 Audit Scope

The following files were in the scope of the audit:

```
src/
├── Create3.sol
├── DeployGate.sol
└── IDeployGate.sol
```

6 Executive Summary

Over the course of 2 days, the Burra Security team conducted an audit on the Centrifuge Create3 Gate smart contracts provided by Centrifuge. In this period, a total of 3 issues were found.

Summary

Project Name	Centrifuge Create3 Gate
Repository	create3-gate
Commit	2f62f3a8cafe...
Audit Timeline	Aug 20th - Aug 21st
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	0
Informational	3
Gas Optimizations	0
Total Issues	3

Summary of Findings

[I-1] Deployment order does not necessarily imply constructor completion	Resolved
[I-2] RLP List-Length Comment Misstates the Payload Size	Resolved
[I-3] Delegate Revocation Leaves Pre-Created Deployment Capabilities Active	Resolved

7 Findings

7.1 Informational

7.1.1 Deployment order does not necessarily imply constructor completion

Target: `src/DeployGate.sol:88`

Severity: Informational

Description: The deployment cursor advances before `CREATE3` executes the current contract's init code. If, during construction, execution reaches an authorized executor that reenters `deploy()`, the next committed deployment can begin before the current constructor finishes.

```
++deployed[validator][id];

target = Create3.deploy(namespaceSalt(validator, salt), initCode);
```

Recommended Mitigation: Consider documenting that deployment order reflects invocation order and does not necessarily guarantee constructor completion order.

Centrifuge: Fixed in centrifuge/create3-gate#5

BurraSec: Fix looks good

7.1.2 RLP List-Length Comment Misstates the Payload Size

Target: `src/Create3.sol:30-36`

Severity: Informational

Description: The implementation correctly derives the proxy's nonce-1 CREATE address with `abi.encodePacked( hex"d694", proxy, hex"01")`, but the preceding comment incorrectly describes `0xd6` as an RLP list of 21 bytes. The list payload is 22 bytes: `0x94` plus the 20-byte proxy form a 21-byte encoded address item, and nonce `0x01` contributes one additional byte. Accordingly, `0xd6 = 0xc0 + 22` is correct and only the comment is wrong. Address derivation and the independent derivation tests are unaffected; this is an informational documentation defect with no demonstrated attack path.

Recommended Mitigation: Change the comment to `0xd6 = RLP list with a 22-byte payload` and retain the independent CREATE-address derivation tests.

Centrifuge: Fixed in centrifuge/create3-gate#5

BurraSec: Fix looks good

7.1.3 Delegate Revocation Leaves Pre-Created Deployment Capabilities Active

Target: `src/DeployGate.sol`

Severity: Informational

Description: `DeployGate.validate()` lets an authorized delegate choose any `id`, salts, init-code hashes, and executors for the validator's namespace. It creates an independent generation under that `(validator, id)` and stores both the executor grants and ordered commitments there (`src/DeployGate.sol:35-63`). A compromised delegate can therefore pre-create commitments under fresh attacker-chosen IDs that are absent from the validator's incident-response list and grant an attacker-controlled executor permission to spend salts that the validator expects to use later.

Calling `setDelegate(delegatee, false)` only clears `isDelegate` and prevents the delegate from making new commitments; it does not invalidate generations, commitments, or executor grants that the delegate created while authorized (`src/DeployGate.sol:67-69`). `deploy()` subsequently checks only the executor and commitment stored under the caller-supplied ID. It does not record or re-check the generation's issuer or the delegate's current

authorization (`src/DeployGate.sol:77-90`). Consequently, after the validator revokes the compromised delegate, an executor named in one of those pre-created generations can still deploy the committed init code.

This becomes protocol-impacting across IDs because `namespaceSalt()` derives the CREATE3 salt from only `(validator, salt)`, excluding `id` (`src/DeployGate.sol:114-120`). A stale attacker-controlled commitment and a later legitimate commitment under a different ID therefore target the same CREATE3 proxy and contract address. If the stale executor deploys first, it can place the delegate-selected code at the expected protocol address and consume the CREATE3 proxy so the legitimate deployment reverts (`src/Create3.sol:12-27`). Replacing a known ID invalidates that ID's old generation, but it does not affect fresh IDs created by the delegate. `Validate` events expose these IDs for complete off-chain reconciliation, yet discovery and per-ID replacement are not an atomic substitute for revocation.

The impact is high when an unspent salt corresponds to a security-critical deterministic protocol address. Likelihood is medium because exploitation requires control of a validator-authorized delegate before revocation, advance creation of a relevant commitment, control of a named executor, an unspent salt, and execution before the validator discovers and replaces the attacker-chosen ID. This is a capability-lifetime and incident-containment failure, not an initial privilege escalation: the delegate was validator-equivalent when it created the commitment and the executor remains limited to the exact init code committed at that time. The assessed repository has no release tags from which to establish an affected or fixed release range.

Recommended Mitigation: Bind each generation to the authority that created it and make revocation invalidate that authority's earlier generations. For example, record the issuing delegate and its current per-validator epoch during `validate()`, increment that delegate's epoch when it is revoked, and require the recorded epoch to remain current in `deploy()`; validator-issued generations can be treated separately. Re-authorizing a delegate must not revive generations from an earlier epoch. Alternatively, add a validator-wide emergency epoch or `revokeAll()` operation that atomically makes every existing ID unreachable. Add regression tests in which a delegate creates commitments under multiple fresh IDs, is revoked, and each named executor is rejected, including a cross-ID same-salt case; also verify that direct validator commitments remain usable and that re-authorization does not reactivate stale grants.

Centrifuge: Fixed in centrifuge/create3-gate#5

BurraSec: Fix looks good, the validator wide term correctly invalidates commitments across all IDs.